

ELO329 - Diseño y Programación Orientados a Objetos

Polimorfismo, ligado dinámico y métodos virtuales en C++

Agustín J. González
Patricio Olivares R.

Universidad Técnica Federico Santa María

Propósito de la clase

En la clase anterior usamos herencia para expresar que un `CElectricMotor` o un `CGasMotor` son tipos de `CMotor`.

Ahora veremos qué ocurre cuando el programa manipula esos objetos mediante punteros o referencias a la clase base.

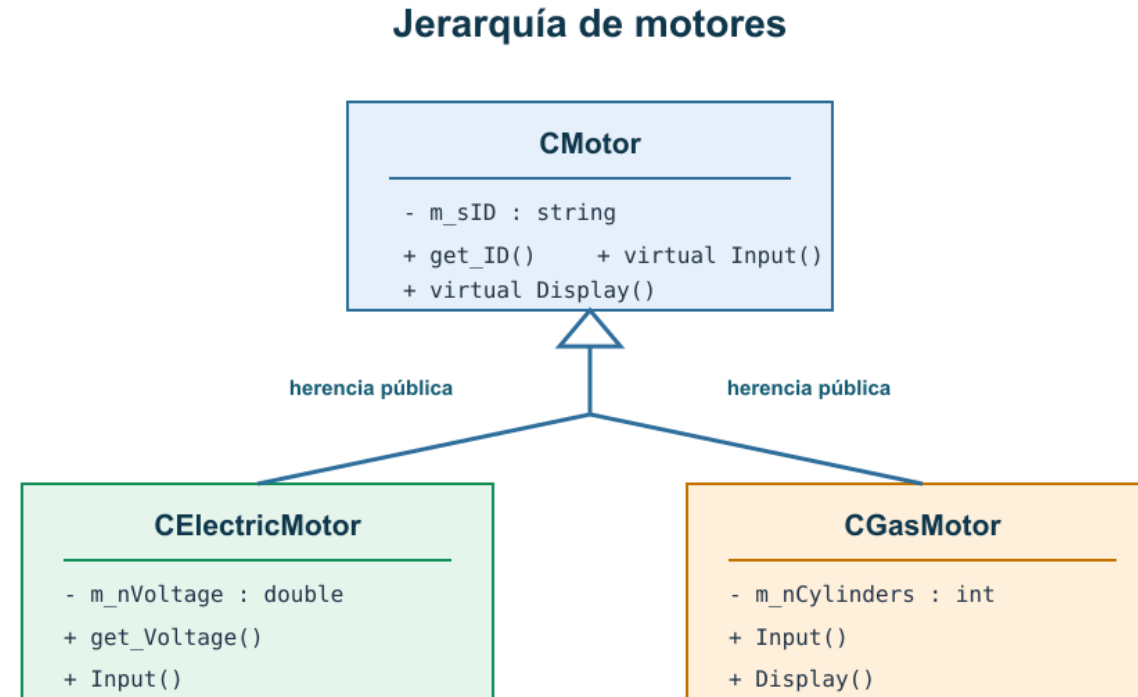
La pregunta central es simple: si el puntero dice ``CMotor*``, pero el objeto real es eléctrico o a gasolina, ¿qué método se ejecuta?

Contenidos

1. Jerarquía de motores
2. Punteros y referencias a clases derivadas
3. Polimorfismo en C++
4. Ligado estático y ligado dinámico
5. Métodos virtuales
6. Vectores de punteros a base
7. Métodos virtuales puros y clases abstractas
8. Declaración incompleta de clases
9. Conversiones de tipo y cast

Jerarquía de clases Motor

Recordemos la jerarquía usada para estudiar herencia.



Interfaz común en la base, comportamiento especializado en las derivadas.

CMotor representa lo común a todos los motores.

Las clases derivadas agregan los datos y operaciones particulares.

Clase base CMotor

```
class CMotor {  
public:  
    CMotor() { }  
    CMotor(const string &id);  
  
    string get_ID() const;  
    void set_ID(const string &s);  
  
    void Display() const;  
    void Input();  
  
private:  
    string m_sID;  
};
```

CMotor administra el identificador común a todos los motores.

Clase CElectricMotor

```
class CElectricMotor : public CMotor {  
public:  
    CElectricMotor();  
    CElectricMotor(const string &id, double volts);  
  
    void Display() const;  
    void Input();  
  
    void set_Voltage(double volts);  
    double get_Voltage() const;  
  
private:  
    double m_nVoltage;  
};
```

El motor eléctrico reutiliza el ID de `CMotor` y agrega voltaje.

Clase CGasMotor

```
class CGasMotor : public CMotor {  
public:  
    CGasMotor();  
    CGasMotor(const string &id, int cylinders);  
  
    void Display() const;  
    void Input();  
  
private:  
    int m_nCylinders;  
};
```

El motor a gasolina reutiliza el ID y agrega número de cilindros.

Puntero de tipo específico

Podemos crear un objeto dinámico usando un puntero de la clase derivada.

```
CElectricMotor *pC = new CElectricMotor;  
  
pC->set_ID("3099999");  
pC->set_Voltage(110.5);  
pC->Display();  
  
delete pC;
```

Aquí no hay sorpresa. El puntero conoce que el objeto es `CElectricMotor`.

La pregunta importante

La herencia pública permite escribir esto:

```
CMotor *pM;  
pM = new CElectricMotor;
```

El puntero está declarado como `CMotor*`.

El objeto creado realmente es un `CElectricMotor`.

La diferencia entre el tipo del puntero y el tipo real del objeto es donde aparece el polimorfismo.

Polimorfismo

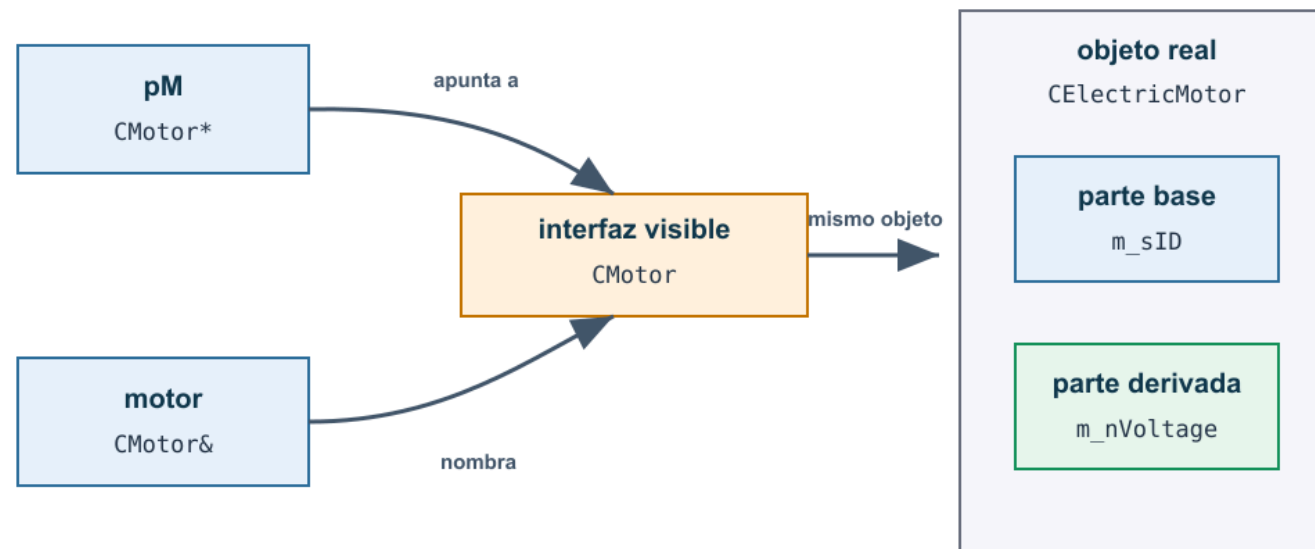
En C++, un puntero o referencia a una clase base puede referirse a una instancia de una clase derivada.

```
CMotor *pM;  
pM = new CElectricMotor;  
  
CElectricMotor em;  
CMotor &motor = em;
```

Esto expresa el principio de sustitución: donde se espera un `CMotor`, puede usarse un subtipo válido.

Punteros y referencias

Tipo declarado y tipo real



El tipo declarado restringe la interfaz. El tipo real importa para las llamadas virtuales.

La referencia no crea una copia. Solo entrega otro nombre para el mismo objeto.

Ligado estático por omisión

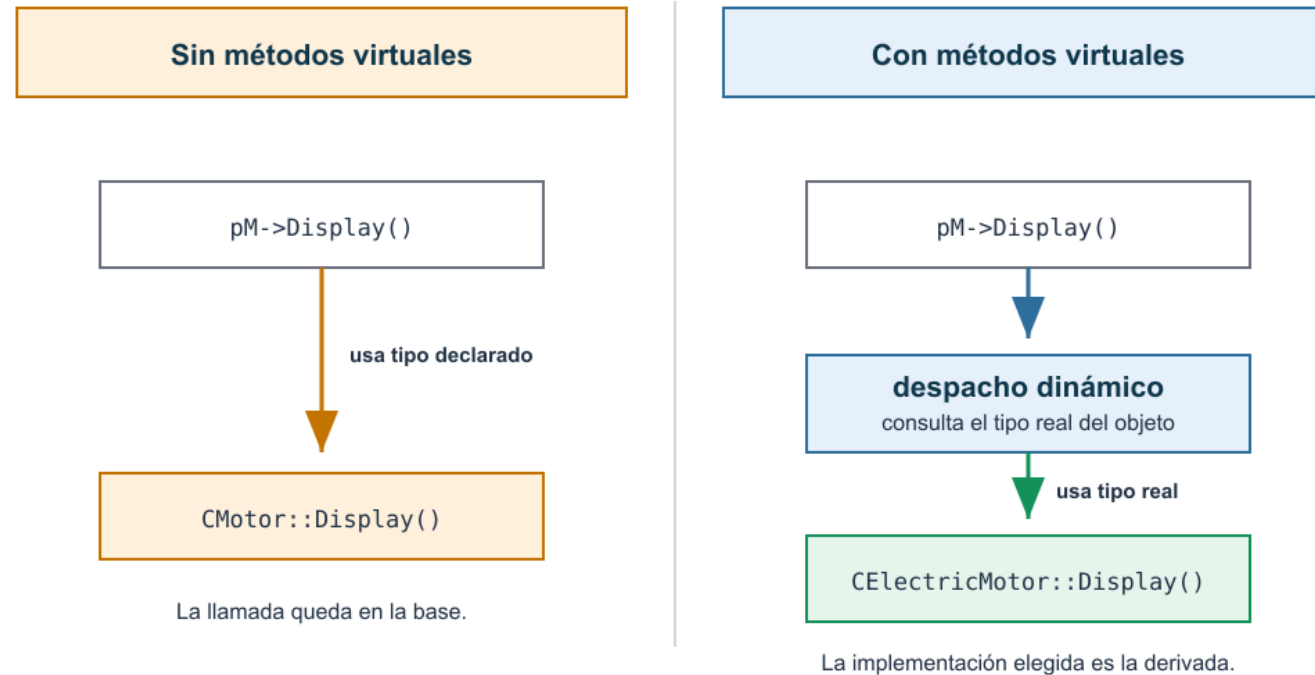
En C++, si el método no es virtual, la llamada se decide usando el tipo declarado del puntero o referencia.

```
CMotor *pM;  
pM = new CElectricMotor;  
  
pM->Input();    // llama a CMotor::Input()  
pM->Display();  // llama a CMotor::Display()
```

Esta es una diferencia importante con Java, donde el despacho dinámico es la regla normal para métodos.

Por qué esto importa

Una llamada, dos decisiones posibles



La expresión puede ser la misma: `pM->Display()` .

Lo que cambia es el criterio usado para elegir la implementación.

Métodos virtuales

Para obtener comportamiento polimórfico, se declara el método como `virtual` en la clase base.

```
class CMotor {  
public:  
    virtual void Display() const;  
    virtual void Input();  
  
private:  
    string m_sID;  
};
```

`virtual` indica que la selección del método debe usar el tipo real del objeto.

Ligado dinámico

Con métodos virtuales, la llamada se resuelve en tiempo de ejecución.

```
CMotor *pM;  
pM = new CElectricMotor;  
  
pM->Input();    // llama a CElectricMotor::Input()  
pM->Display();  // llama a CElectricMotor::Display()
```

El tipo del puntero sigue siendo `CMotor*`.

La ejecución usa la versión correspondiente al objeto apuntado.

Mantener la intención en derivadas

El material original vuelve a marcar estos métodos como virtuales en las clases derivadas.

En C++ moderno, `override` expresa mejor la intención.

```
class CElectricMotor : public CMotor {  
public:  
    void Display() const override;  
    void Input() override;  
};
```

Si la firma no coincide con la de la clase base, el compilador reporta el error.

Implementación de CElectricMotor

```
void CElectricMotor::Input() {  
    cout << "[ElectricMotor]: ";  
    CMotor::Input();  
  
    double volts;  
    cout << "Voltage: ";  
    cin >> volts;  
    set_Voltage(volts);  
}
```

Primero se lee la parte común mediante `CMotor::Input()`.

Luego se lee el dato propio del motor eléctrico.

Display de CElectricMotor

```
void CElectricMotor::Display() const {  
    cout << "[ElectricMotor] ";  
    CMotor::Display();  
    cout << ", Voltage=" << m_nVoltage;  
}
```

La clase derivada no duplica el manejo del ID.

Reutiliza el método base y agrega el voltaje.

Implementación de CGasMotor

```
void CGasMotor::Input() {  
    cout << "[GasMotor]: ";  
    CMotor::Input();  
    cout << "Enter the number of cylinders: ";  
    cin >> m_nCylinders;  
}  
  
void CGasMotor::Display() const {  
    cout << "[GasMotor] ";  
    CMotor::Display();  
    cout << ", Cylinders=" << m_nCylinders;  
}
```

La misma interfaz pública produce comportamientos distintos.

Función que recibe un puntero a base

```
void GetAndShowMotor(CMotor *pC) {  
    pC->Input();  
    cout << "Here's what you entered:\n";  
    pC->Display();  
    cout << "\n\n";  
}
```

La función no necesita conocer si recibe un motor eléctrico, a gasolina o base.

Solo requiere que el objeto responda a la interfaz de `CMotor` .

Versión con referencia

La misma idea puede escribirse con una referencia.

```
void GetAndShowMotor(CMotor &m) {  
    m.Input();  
    cout << "Here's what you entered:\n";  
    m.Display();  
    cout << "\n\n";  
}
```

En ambos casos, el ligado dinámico ocurre si los métodos son virtuales.

Llamados con distintos tipos

```
CGasMotor *pG = new CGasMotor;  
GetAndShowMotor(pG);  
  
CElectricMotor *pE = new CElectricMotor;  
GetAndShowMotor(pE);  
  
CMotor *pM = new CGasMotor;  
GetAndShowMotor(pM);
```

Cada llamada usa la misma función.

El método ejecutado depende del objeto real.

Salida esperada

```
[GasMotor]: Enter the Motor ID: 234323  
Enter the number of cylinders: 3
```

Here's what you entered:

```
[GasMotor] ID=234323, Cylinders=3
```

```
[ElectricMotor]: Enter the Motor ID: 234324  
Voltage: 220
```

Here's what you entered:

```
[ElectricMotor] ID=234324, Voltage=220
```

La interfaz común no elimina la identidad concreta del objeto.

Vector de punteros a motores

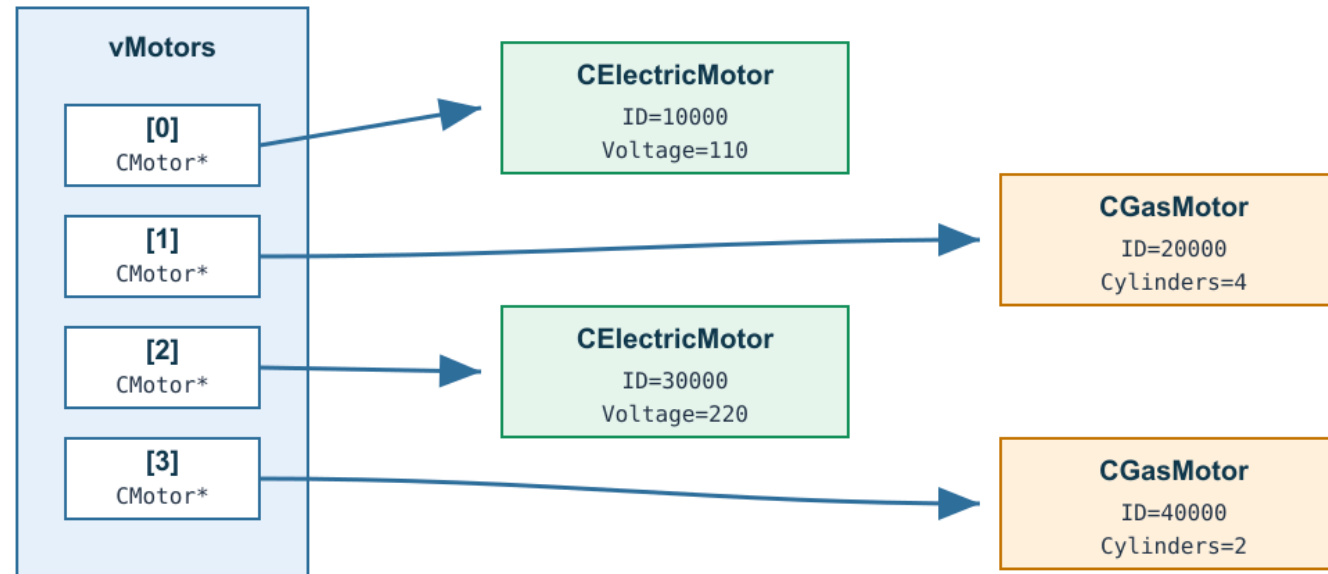
Un `vector<CMotor*>` puede almacenar punteros a cualquier objeto derivado de `CMotor`.

```
vector<CMotor*> vMotors;  
CMotor *pMotor;  
  
pMotor = new CElectricMotor("10000", 110);  
vMotors.push_back(pMotor);  
  
pMotor = new CGasMotor("20000", 4);  
vMotors.push_back(pMotor);
```

El contenedor guarda direcciones con tipo base.

Un vector heterogéneo

`vector<CMotor*>` como colección polimórfica



Un solo tipo de puntero permite apuntar a objetos de clases distintas.

Esta es una forma común de construir colecciones polimórficas en C++.

Mostrar el vector

```
void ShowVector(const vector<CMotor*> &vMotors) {  
    cout << "---- Vector of Motor Pointers ----\n";  
  
    for (int i = 0; i < vMotors.size(); i++) {  
        cout << (i + 1) << ": ";  
        vMotors[i]->Display();  
        cout << endl;  
    }  
}
```

ShowVector no pregunta por la clase concreta.

La llamada virtual hace el trabajo.

Salida de ShowVector

```
----- Vector of Motor Pointers -----  
1: [ElectricMotor] ID=10000, Voltage=110  
2: [GasMotor] ID=20000, Cylinders=4  
3: [ElectricMotor] ID=30000, Voltage=220  
4: [GasMotor] ID=40000, Cylinders=2
```

El vector es uniforme para el compilador.

El comportamiento es específico para cada objeto.

Liberación de almacenamiento

Los objetos creados con `new` deben liberarse.

```
for (int i = 0; i < vMotors.size(); i++) {  
    delete vMotors[i];  
}
```

Esto elimina cada objeto apuntado por el vector.

El vector solo almacenaba direcciones, no era dueño automático de los objetos.

Destructor virtual

Si una clase base se usará de forma polimórfica y se hará `delete` mediante `CMotor*`, el destructor de la base debe ser virtual.

```
class CMotor {  
public:  
    virtual ~CMotor() = default;  
    virtual void Display() const;  
    virtual void Input();  
};
```

Sin destructor virtual, destruir un objeto derivado mediante un puntero a base es un error de diseño en C++.

delete para arreglos

No se debe confundir borrar objetos individuales con borrar arreglos.

```
CMotor *motor = new CMotor[40];  
delete[] motor;
```

Para un solo objeto se usa `delete` .

Para un arreglo creado con `new[]` se usa `delete[]` .

Métodos virtuales puros

Un método virtual puro no tiene implementación en la clase base.

En C++ se declara con `= 0`.

```
class CMotor {  
public:  
    virtual ~CMotor() = default;  
    virtual void Display() const = 0;  
    virtual void Input() = 0;  
};
```

La clase derivada debe implementar esos métodos para poder crear objetos.

Clases abstractas

Una clase con uno o más métodos virtuales puros es una clase abstracta.

No se requiere escribir una palabra `abstract`.

```
CMotor m; // error  
CMotor *pM = new CMotor; // error
```

Sí se pueden usar punteros y referencias a la clase abstracta.

```
CMotor *pM = new CElectricMotor;
```

Ejemplo CatsAndDogs

El ejemplo de la carpeta usa una clase base abstracta.

```
class Animal {  
public:  
    virtual ~Animal() = default;  
    virtual void print() = 0;  
};  
  
class Cat : public Animal {  
public:  
    Cat(int i);  
    void print() override;  
};  
  
class Dog : public Animal {  
public:  
    Dog(int i);  
    void print() override;  
};
```

Ejemplo CatsAndDogs

`Animal` define la interfaz. `Cat` y `Dog` definen el comportamiento concreto.

Polimorfismo con Animal

```
vector<Animal*> animals;

for (int i = 0; i < 7; i++) {
    animals.push_back(new Cat(i));
}

animals.push_back(new Dog(7));

for (int i = 0; i < animals.size(); i++) {
    animals[i]->print();
}
```

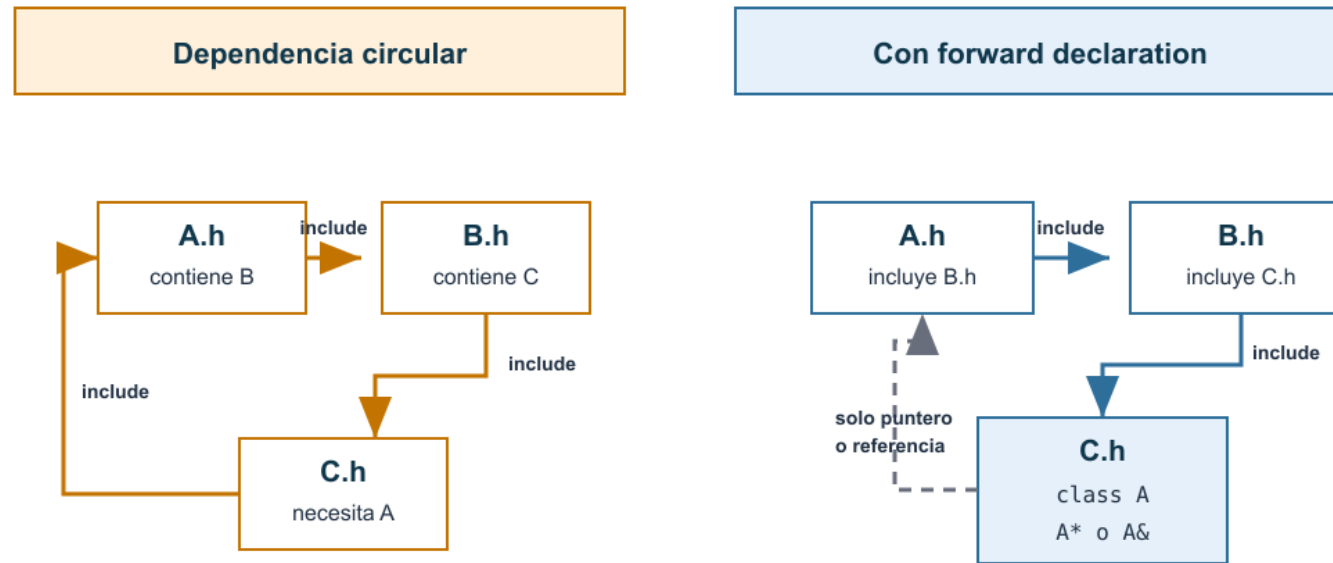
El patrón es el mismo que con `CMotor*`.

El vector almacena punteros a la clase base.

Declaración incompleta de clases

En algunos diseños aparecen dependencias circulares entre encabezados.

Declaración incompleta para cortar un ciclo



La declaración incompleta evita incluir A.h cuando basta conocer que la clase existe.

Si cada encabezado incluye al siguiente, el preprocesador puede entrar en un ciclo de inclusiones.

Dependencia circular

```
// A.h
#include "B.h"
class A {
private:
    B b;
};

// B.h
#include "C.h"
class B {
private:
    C c;
};
```

A y B contienen objetos por valor.

Por eso el compilador necesita conocer el tamaño completo de B y C.

Forward declaration

En `c.h` se puede declarar que `A` existe, sin incluir su definición completa.

```
// c.h
class A;

class C {
private:
    A &a;
};
```

Esto funciona porque una referencia tiene tamaño conocido.

El compilador no necesita conocer todos los atributos de `A` para reservar espacio para `A&`.

Cuándo se puede usar

Una declaración incompleta sirve para atributos que son punteros o referencias.

```
class A;  
  
class C {  
private:  
    A *a;  
};
```

No sirve para contener un objeto por valor.

```
class A;  
  
class C {  
private:  
    A a; // error  
};
```

El compilador necesita saber cuánto ocupa `A`.

Conversiones implícitas

C++ realiza conversiones automáticas entre tipos numéricos intrínsecos.

```
int n = 26;  
double x = n;  
double y = 36;  
int b = x;  
bool isOK = 1;  
int m = true;  
char ch = 26;  
int w = 'A';
```

Algunas conversiones pueden producir advertencias si hay pérdida de información.

Cast tradicional y estilo función

Un cast convierte explícitamente un dato de un tipo a otro.

```
int n = (int)3.5;  
int w = int(3.5);  
bool isOK = bool(15);  
char ch = char(86);  
string st = string("123");
```

En tipos básicos, se usa muchas veces para dejar clara una conversión que el compilador podría hacer.

Conversiones no disponibles

No toda expresión puede convertirse solo por escribir un tipo.

```
int x = int("123");    // error  
string s = string(3.5); // error
```

Un cast no inventa una transformación que no existe.

También puede usarse para interpretar bytes, aunque no es una técnica recomendable para código normal.

```
double x = 3.1415;  
char *p = (char*)&x;
```

static_cast

`static_cast` reemplaza al estilo función cuando la conversión es conocida en compilación.

```
int w = static_cast<int>(3.5);  
bool isOK = static_cast<bool>(1);  
char ch = static_cast<char>(86);
```

No revisa si una conversión entre clases es segura en tiempo de ejecución.

Por eso es rápido, pero exige que el programador sepa lo que está haciendo.

dynamic_cast

`dynamic_cast` se usa con punteros o referencias a objetos dentro de una jerarquía polimórfica.

```
class CBase {  
public:  
    virtual ~CBase() = default;  
};  
  
class CDerived : public CBase { };
```

Permite revisar si el objeto real es compatible con el nuevo tipo.

No hay un operador equivalente directo a `instanceof` de Java.

Ejemplo de dynamic_cast

```
CBase b;  
CBase *pb;  
CDerived d;  
CDerived *pd;  
  
pb = dynamic_cast<CBase*>(&d);    // ok  
pd = dynamic_cast<CDerived*>(&b); // retorna null
```

Si el destino es puntero y no hay compatibilidad, el resultado es `nullptr`.

Si el destino es referencia y falla, se lanza una excepción.

reinterpret_cast

`reinterpret_cast` permite conversiones de bajo nivel entre tipos no relacionados.

```
double x = 3.1415;  
char *p = reinterpret_cast<char*>(&x);
```

La operación reinterpreta el valor binario.

No se recomienda su uso en este curso, salvo que se esté estudiando representación de datos o interfaces de muy bajo nivel.

const_cast

`const_cast` permite quitar o agregar `const` en una expresión.

```
void foo(char *str) {  
    // str no se trata como constante  
}  
  
const char *pHola = "Hola";  
foo(const_cast<char*>(pHola));
```

Debe usarse con mucho cuidado.

Si el objeto original realmente es constante y la función intenta modificarlo, el programa queda en un estado inválido.

Comparación de casts

Cast	Uso principal	Riesgo
C style	Compatibilidad con C	Oculto demasiado
<code>static_cast</code>	Conversiones conocidas	No revisa el tipo real
<code>dynamic_cast</code>	Jerarquías polimórficas	Requiere clase base polimórfica
<code>reinterpret_cast</code>	Bajo nivel	Muy propenso a errores
<code>const_cast</code>	Ajustar <code>const</code>	Puede romper contratos

La regla práctica es usar el cast más específico para expresar la intención.

Ideas clave

- Un puntero o referencia a base puede referirse a un objeto derivado.
- En C++, las llamadas no virtuales se deciden por el tipo declarado.
- `virtual` habilita ligado dinámico usando el tipo real del objeto.
- Una función que recibe `CMotor*` puede trabajar con motores distintos.
- Un `vector<CMotor*>` permite colecciones polimórficas.
- Si hay `delete` mediante puntero a base, el destructor debe ser virtual.
- Un método virtual puro convierte la clase en abstracta.
- Una declaración incompleta sirve con punteros y referencias.
- Los cast de C++ expresan distintos niveles de seguridad y responsabilidad.